Exemplul 1)
intrerupator.smv

```
-----------------
MODULE main
VAR
        v: {off, on};
ASSIGN
        next(v):=
                case
                        v = on : off;
                        v = off : on;
                esac;

LTLSPEC
F(v = on)
LTLSPEC
F(v = off)
LTLSPEC
G(v = on -> X (v = off))
LTLSPEC
G(v = off -> X (v = on))
LTLSPEC
G(v = on | v = off)
LTLSPEC
G(!(v = off & v = on))
LTLSPEC
G(F(v=on))
LTLSPEC
G(F(v=off))
LTLSPEC
G(v=on)
LTLSPEC
F(G(v=on))
LTLSPEC
```

```
X(v=on)
----------------------------

Exemplul 2)semafor.smv
MODULE main
VAR
 state: {green, yellow, red};
 redBefore : boolean;
ASSIGN
 init(state) := red;
 init(redBefore) := TRUE;
 next(state) := case
  state = red : yellow;
  state = yellow & !redBefore : red;
  state = yellow & redBefore : green;
  state = green : yellow;
  TRUE : state;
 esac;
 next(redBefore):= case
  state = red: TRUE;
  state = green : FALSE;
  TRUE : redBefore;
 esac;

SPEC AF (state = green);
SPEC AF (state = red);
SPEC AF (state = yellow);
SPEC AG (state = green -> AF (state = red));
SPEC AG (state = red -> AF (state = green));
SPEC AG (!(state = red) | !(state = green));
SPEC AG AF (state = green);
SPEC AG AF (state = yellow);
SPEC AG AF (state = red);
SPEC AG (state = red -> AX (state = yellow));
```

```
SPEC AG (state = green -> AX (state =
yellow));
SPEC AG (state = green -> AX (!(state =
red)));
SPEC AG (state = red -> AX (!(state =
green)));
SPEC AG (state = yellow -> AX (state = red |
state = green));
SPEC AG (state = red ->(AX (state = yellow) &
AX (AX (state = green))));
SPEC AG (state = green ->(AX (state = yellow)
& AX (AX (state = red))));
SPEC AG ((state = red ->(AX (state = yellow)
& AX (AX (state = green)))) & (state = green
->(AX (state = yellow) & AX (AX (state =
red))))) ;
SPEC AG (state = yellow -> AX(state =
green));
-----------------------------

Exemplul 3)vehicul.smv
MODULE main
VAR
        command : {go_up, go_down, go_left,
go_right, do_nothing};
        state :
{left_up,left_down,right_up,right_down};
ASSIGN

--init(state) := left_up;

next(state) := case
        state = left_up & command = go_up:
left_up;
```

```
        state = left_up & command = go_down:
left_down;
        state = left_up & command = go_left:
left_up;
        state = left_up & command = go_right:
right_up;
        state = left_up & command =
do_nothing: left_up;
        state = left_down & command = go_up:
left_up;
        state = left_down & command =
go_down: left_down;
        state = left_down & command =
go_left: left_down;
        state = left_down & command =
go_right: right_down;
        state = left_down & command =
do_nothing: left_down;
        state = right_up & command = go_up:
right_up;
        state = right_up & command = go_down:
right_down;
        state = right_up & command = go_left:
left_up;
        state = right_up & command =
go_right: right_up;
        state = right_up & command =
do_nothing: right_up;
        state = right_down & command = go_up:
right_up;
        state = right_down & command =
go_down: right_down;
        state = right_down & command =
go_left: left_down;
```

```
        state = right_down & command =
go_right: right_down;
        state = right_down & command =
do_nothing: right_down;
esac;

SPEC
        AG((state = left_up & command =
go_down) -> AX(state = left_down))
SPEC
        AG((state = left_up & command =
go_right) -> AX(state = right_up))
SPEC
        AG((state = left_up & (command =
go_up | command = go_left | command =
do_nothing)) -> AX(state = left_up))

SPEC
        AG((state = left_down & command =
go_up) -> AX( state = left_up))
SPEC
        AG((state = left_down & command =
go_right) -> AX( state = right_down))
SPEC
        AG((state = left_down & (command =
go_down | command = go_left | command =
do_nothing)) -> AX(state = left_down))

SPEC
        AG((state = right_up & command =
go_down) -> AX(state = right_down))
SPEC
        AG((state = right_up & command =
go_left) -> AX(state = left_up))
```

```
SPEC
        AG((state = right_up & (command =
go_up | command = go_right | command =
do_nothing)) -> AX(state = right_up))

SPEC
        AG((state = right_down & command =
go_up) -> AX(state = right_up))
SPEC
        AG((state = right_down & command =
go_left) -> AX(state = left_down))
SPEC
        AG((state = right_down & (command =
go_down | command = go_right | command =
do_nothing)) -> AX(state = right_down))
SPEC
        AG(state = right_down -> AX(state =
left_up))
--------------------------
Exemplul 4.1)mutexCTL.smv
MODULE main
VAR
  semaphore : boolean;
  proc1 : process user(semaphore);
  proc2 : process user(semaphore);
ASSIGN
  init(semaphore) := FALSE;
SPEC
  AG! (proc1.state = critical & proc2.state =
critical)
SPEC
  AG (proc1.state = entering -> AF
proc1.state = critical)
```

```
MODULE user(semaphore)
VAR
  state : {idle,entering,critical,exiting};
ASSIGN
  init(state) := idle;
  next(state) :=
    case
      state = idle : {idle,entering};
      state = entering & !semaphore :
critical;
      state = critical : {critical,exiting};
      state = exiting : idle;
      TRUE : state;
    esac;
  next(semaphore) :=
    case
      state = entering : TRUE;
      state = exiting : FALSE;
      TRUE : semaphore;
    esac;
FAIRNESS
  running
-------------------------------
Exemplul 4.2) mutexLTL.smv
MODULE main
VAR
        semaphore : boolean;
        proc1 : process user(semaphore);
        proc2 : process user(semaphore);
ASSIGN
        init(semaphore) := FALSE;

LTLSPEC G ! (proc1.state = critical &
proc2.state = critical)
```

```
LTLSPEC G (proc1.state = entering -> F
proc1.state = critical)

MODULE user(semaphore)
VAR
        state : {idle, entering, critical,
exiting};
ASSIGN
        init(state) := idle;
        next(state) :=
                case
                        state = idle : {idle,
entering};
                        state = entering &
!semaphore : critical;
                        state = critical :
{critical, exiting};
                        state = exiting :
idle;
                        TRUE : state;
                esac;
        next(semaphore) :=
                case
                        state = entering :
TRUE;
                        state = exiting :
FALSE;
                        TRUE : semaphore;
                esac;

FAIRNESS
running
```